PROJETO DE ESTUDO · 2026

# Próspero

Um simulador de bolsa com quatro microsserviços, construído em 13 dias com um fluxo de agentes de IA, gates de CI e um humano definindo os fluxos e validando a aplicação.

Java 26 · Quarkus | Kafka | PostgreSQL | Angular | Go

SAGA · Outbox | Prometheus · Grafana | Claude Code · Codex

POR QUE EXISTE

# Por que o Próspero existe

Projeto de estudo escolhido estrategicamente por apresentar **alta complexidade vertical e baixa complexidade horizontal**, permitindo explorar, experimentar e evoluir diferentes abordagens arquiteturais sem a necessidade de um sistema de grande escala.

O objetivo era desenvolver minhas habilidades em arquitetura de software e experimentar um fluxo de desenvolvimento orientado por IA, no qual a LLM assume a geração de código. A proposta priorizava **mínima burocracia, baixo consumo de tokens, qualidade, segurança e autonomia da IA**, mesmo que isso significasse um desenvolvimento mais lento, mantendo a independência de modelos específicos.

- Definição da arquitetura, roadmap, escopo, requisitos funcionais e não funcionais e stack tecnológica do projeto.
- Estruturação de uma **governança de desenvolvimento orientada por IA**, estabelecendo um fluxo de trabalho flexível, com baixa burocracia e **gates de qualidade** para garantir segurança, consistência arquitetural, testes e qualidade do código gerado por LLMs, preservando a autonomia dos agentes e a independência de modelos específicos.
- Adoção de **TDD (Test-Driven Development)** para garantir a aderência às regras de negócio e preservar a integridade do domínio da aplicação.
- Implementação de **pipelines automatizados com gates de qualidade**, incluindo revisões arquiteturais, análise de código, testes e verificações de segurança.
- Utilização de **MCP (Model Context Protocol)** e mecanismos de memória compartilhada para garantir a continuidade do desenvolvimento entre sessões e diferentes agentes de IA, reduzindo a perda de contexto, o retrabalho e o consumo desnecessário de tokens, além de promover consistência nas decisões técnicas e arquiteturais.

APRENDIZADOS

# O que deu certo e o que temos que melhorar

## Deu certo

- **Autonomia da LLM, o maior benefício:** gerou código coeso, com o contexto do projeto e da arquitetura, e com qualidade.
- **Gates de CI** e revisores separados pegaram falhas que os testes unitários não viam.
- **TDD e ArchUnit** mantiveram o domínio fiel às regras de negócio.
- **HLDs e ADRs** deram à IA um pedido escrito.
- **Memória e skills** permitiram alternar entre Claude Code e Codex sem perder contexto.
- **Ritmo:** 69 PRs em 13 dias, nenhum descartado.

## A melhorar

- **Burocracia:** o trio de validação e os três gates custam tempo e tokens. Falta calibrar quando cada um é necessário.
- **Validação tardia:** 4 erros passaram pelos gates e só foram identificados na minha validação humana, depois da entrega.
- **Latência:** a meta da boleta ao negócio segue não cumprida.

O QUE ELE SIMULA

# Como o preço nasce do encontro entre ordens

Cinco ações fictícias, um livro de ofertas por ativo e o caminho completo de uma ordem, da boleta à liquidação.

- **Ordens** Market, Limit, Stop, Stop Limit e OCO, com validade Dia, IOC e FOK, lote, tick e túnel de preço.
- **Risco** com reserva de saldo e custódia antes de a ordem chegar ao livro.
- **Matching** por prioridade preço-tempo, com execuções parciais e preço do agente passivo.
- **Pós-negociação:** posição, preço médio, PnL realizado e não realizado, razão em partidas dobradas e liquidação em D+2.
- **Cancelamento** com liberação de saldo e **trajetória da ordem em tempo real** na tela.
- Um **robô formador de mercado** e uma **API de carga** em Go para estressar o sistema.

EM NÚMEROS

# De 16 a 28 de outubro

**69**
pull requests integrados, com revisão, segurança e testes na CI

**358**
commits na `main`

**4**
microsserviços, cada um com o próprio PostgreSQL

**18**
ADRs registrando decisões e o custo aceito em cada uma

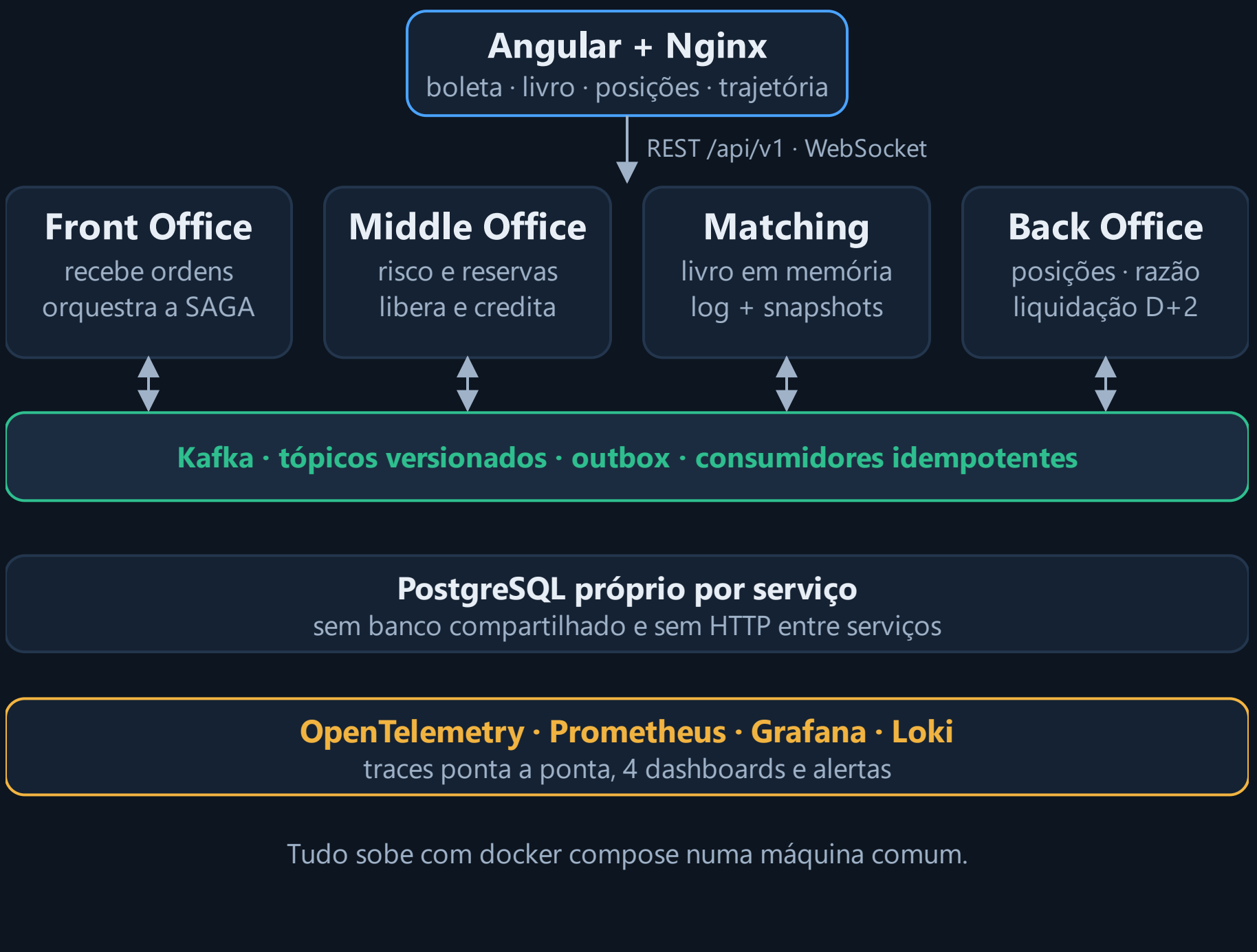
**~65 mil**
linhas de Java, TypeScript, Go, Python e SQL

**174**
arquivos de teste Java (JUnit, REST Assured, ArchUnit), além de E2E Playwright

Números conferidos no Git e no GitHub

ARQUITETURA

# Quatro serviços que só conversam por Kafka




Tudo sobe com docker compose numa máquina comum.

CONSISTÊNCIA

# A vida de uma ordem, sem perder um centavo

1 **Boleta → Front Office**
Chave de idempotência, validação de lote, tick e túnel. A ordem nasce `RECEBIDA` e o comando sai pela outbox, publicado só depois do commit.

2 **Middle Office reserva**
Saldo ou custódia ficam presos antes do livro. Resposta: `RESERVADA` ou `REJEITADA`.

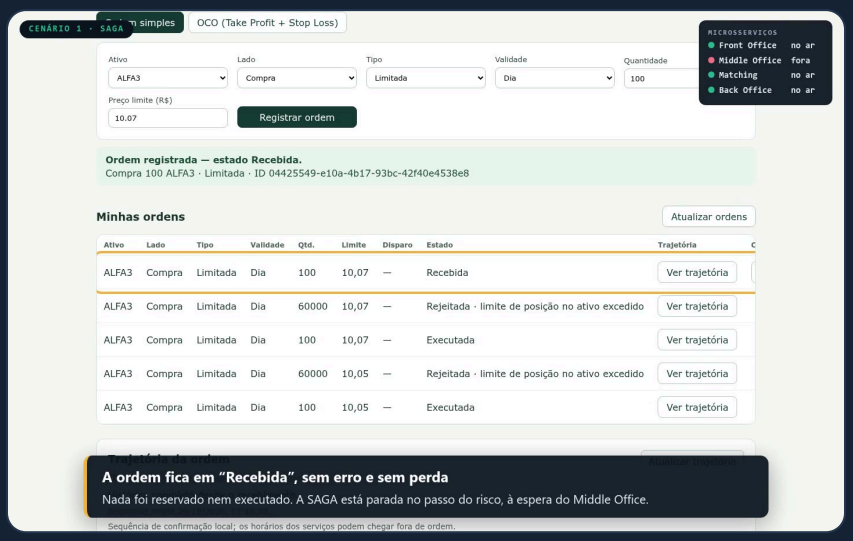
3 **Matching executa**
Escritor único por instância e log de eventos. O replay gera os mesmos bytes e os mesmos ids.

4 **Back Office registra**
Negócio, posição, razão balanceada e liquidação D+2 com crédito confirmado no MO.

5 **Falhas ensaiadas**
Serviço derrubado no meio da carga, mensagem repetida, cancelamento contra execução: a SAGA compensa e as reservas fecham.

VEJA FUNCIONANDO

# O sistema rodando de verdade




VÍDEO 1 · 2 MIN

## SAGA: sucesso, recusa e falha

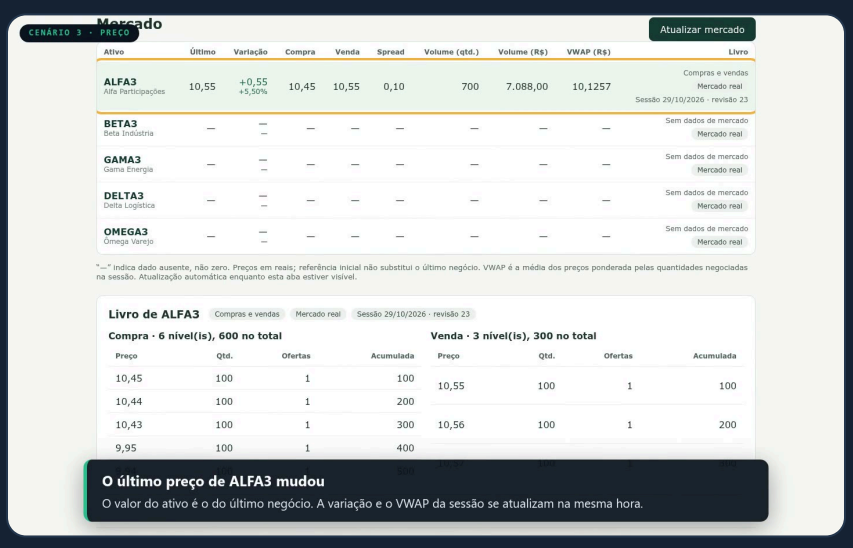
Uma ordem executada, outra recusada pelo risco e o Middle Office derrubado no meio do fluxo. A ordem espera em “Recebida” e é executada quando o serviço volta.




VÍDEO 2 · 2 MIN

## Grafana, alertas e traces

Painéis de ordens, risco, matching e back office, as regras de alerta e um trace do Jaeger atravessando os quatro serviços.




VÍDEO 3 · 1,5 MIN

## Mudar o preço de um ativo

Uma variável reposiciona o livro de ALFA3 em R$ 10,50, o próximo negócio define o novo preço e o túnel de ±10% barra um salto para R$ 13,00.

DECISÕES TÉCNICAS

# Escolhas que deixaram o sistema verificável

### Nada de double

Valores em decimal exato e racionais; o ArchUnit impede `double` no domínio.

### Contratos versionados

Mensagens Kafka com fixtures e SHA-256. Os quatro serviços testam contra os mesmos bytes.

### Outbox após commit

Nenhum evento sai de uma transação que ainda pode desfazer.

### Livro determinístico

Matching em memória com log e snapshots; reiniciar dá o mesmo resultado.

### Testes em planilha

Cenários de negócio em XLSX executados contra a API real, legíveis por quem não programa.

### Carga em Go

API de carga própria com preflight de recursos e evidência consolidada em JSON.

REQUISITOS NÃO FUNCIONAIS

# Medido, inclusive o que não passou

| | | |
|---|---|---|
| **Precisão** | decimal exato, ArchUnit | **comprovado** |
| **Determinismo** | replay com mesmos bytes e ids | **comprovado** |
| **Ordenação** | escritor único, sequência global | **comprovado** |
| **Reservas** | SAGA e compensação sob falha | **comprovado** |
| **Idempotência** | reentrega sem efeito duplo | **comprovado** |
| **Observabilidade** | traces, painéis, alertas | **comprovado** |
| **Ambiente local** | compose com limites de recurso | **comprovado** |
| **Latência do matching** | p99 0,61 ms | **cumprido** |
| **Boleta → negócio** | p50 35 ms · p99 223,7 ms | **não cumprido** |

> **Por que publicar o que falhou?** A medição dedicada (1000 amostras) mostrou a cauda no caminho até o matching. O veredito ficou registrado como "medido, não cumprido", em vez de ajustar a meta.

Medição em WSL2, 12 CPUs, 8 GB

QUEM CONDUZIU

# A IA escreveu o código. As decisões foram minhas.

Sou o responsável pelo projeto. Antes de a IA escrever uma linha, defini como ela devia trabalhar, e depois conferi o que ela entregou.

- **Revisei cada PR** e só eu autorizei os merges. A IA abria o PR com evidências; a aprovação final era humana.
- **Padrões do projeto:** REST → Actor → domínio, DDD, SAGA, Object Calisthenics, DTOs em `/api/v1` e nada de `double` em valor financeiro.
- **Tecnologias:** Java 26 com Quarkus, Kafka, PostgreSQL por serviço, Angular, Go e a pilha de observabilidade.
- **Arquitetura por HLD e ADR:** 6 HLDs desenham o ecossistema e cada serviço; cada escolha virou ADR com alternativas e custo aceito.
- **Fluxo de desenvolvimento e roadmap:** refinar, planejar, TDD, validar em trio e três gates de CI, com as tarefas em ordem de entrega.
- **Agentes especializados:** desenhei os seis papéis e a regra de que quem implementa não aprova o próprio trabalho.

FLUXO DE IA

# Um time de agentes com papéis separados

A sessão principal (o **Oráculo**) orquestra. Quem implementa nunca aprova o próprio trabalho.

ARQUITETURA E REVISÃO

## Yoda

HLDs, ADRs, fronteiras de módulo e auditoria de PR.

QA

## Patrick Jane

Matriz de testes, RED/GREEN, idempotência dos testes de integração.

SEGURANÇA E REDES

## Neo

Diff, credenciais, dependências, CI e investigação ponta a ponta.

IMPLEMENTAÇÃO

## Severino

Backend, frontend, migrações e testes, seguindo o desenho aprovado.

SRE / DEVOPS

## Jarvis

Ambientes, gates operacionais, observabilidade e incidentes.

DIAGRAMAS

## C4

Diagramas C4 só com fontes aprovadas, sem inventar topologia.

FLUXO DE IA

# O caminho de cada tarefa do roadmap

1 **Refinar**

Lê PRD, HLD, ADRs e a memória do projeto; colhe pareceres de arquitetura, QA e segurança antes de escrever código.

2 **Planejar e TDD**

Plano em passos pequenos; cada teste falha pelo motivo certo antes da implementação.

3 **Validar em trio**

Revisão, QA e segurança em três agentes separados de quem implementou.

4 **PR e três gates de CI**

Revisão, segurança (gitleaks e Trivy) e testes com o sistema inteiro de pé, carga e E2E incluídos.

5 **Merge com autorização humana**

A IA não faz merge, release ou ação destrutiva sem autorização explícita.

FLUXO DE IA

# Memória, skills e guarda-corpos

- **Memória persistente (ai-memory):** decisões e contexto sobrevivem entre sessões e ferramentas. Tudo o que vem da memória é tratado como histórico e conferido com o Git antes de virar ação.
- **Skills versionadas:** refinar tarefa, TDD, diagnosticar CI, depurar sistematicamente, estado do roadmap. São procedimentos repetíveis, sem depender de prompt improvisado.
- **Hooks:** gitleaks bloqueia push com segredo; a configuração de IA se sincroniza com um repositório próprio ao fim da sessão.
- **Proporcionalidade:** agentes só para mudanças maiores; documentação e ajustes pontuais seguem direto, com checagens determinísticas.
- **Humano no comando:** escopo, decisões de produto e merges ficaram comigo.

RESULTADO

# Fiel ao que pedi, rápido e com o que faltou à vista

### Fidelidade

Os padrões, HLDs e ADRs foram seguidos e o ArchUnit barrava desvio na CI. Nenhum merge, release ou ação destrutiva sem a minha autorização.

### Rapidez

69 PRs em 13 dias, cerca de cinco por dia. Na mediana, cerca de 2 h entre abrir o PR e o merge.

### Eficácia

8 dos 9 requisitos não funcionais comprovados ou cumpridos. Nenhum PR descartado.

### Onde precisou de mim

Os agentes revisores acharam divergências entre código e documentação, e a minha validação depois das entregas gerou duas rodadas de ajustes.

> **Saldo:** a IA fez o que foi pedido quando o pedido estava escrito em padrão, HLD, ADR e roadmap. Onde a regra não estava clara, a revisão humana e a CI fizeram a diferença.

O QUE APRENDI

# A CI de verdade pega o que o teste unitário não vê

- O cenário de cancelamento sob carga passava em todos os testes unitários e falhou várias vezes na CI: porta que muda depois do restart, cabeçalho de idempotência ausente e um caminho de recusa legítimo que o script não previa.
- Um parser do Kafka quebrava com - no lag. Só apareceu rodando contra o broker real.
- Revisores separados de quem implementou acharam divergências entre código e documentação que o implementador não tinha visto.
- Registrar "não cumprido" vale mais do que um número bonito sem medição.

> **Código, ADRs, HLDs e roadmap estão abertos:**
> `github.com/mclovin137/Prospero`